Vision-Only Reactive Corridor Navigation for a Quadruped using a Frozen Velocity Policy and a Hard Passage Gate

Mohammadreza AhmadiTeshnizi · Patrik Perčinić — *Team Ava*

Robotics 2026, LIACS — Leiden University

video: github.com/teshnizi2/go2-ball-follower/blob/main/demo/go2_submission.mp4 | code: github.com/teshnizi2/go2-ball-follower

Abstract— We present a system in which a simulated Unitree Go2 quadruped follows a moving red ball through a cluttered, never-ending corridor using *only* its head-mounted RGB camera — no GPS, lidar, depth sensor, or pre-built map. We wrap a *frozen*, pre-trained PPO velocity-tracking locomotion policy with three layers we built ourselves: an HSV + CamShift perception module, a reactive free-band planner, and a difficulty curriculum. Our central observation is that the policy's lateral-velocity (v_y) command channel — left unused by a naïve forward-chasing controller — can be driven so that an otherwise unicycle-like walker can *strafe* into narrow wall-side passages without turning. Coupling this with a *hard passage gate* (the robot may not cross a row until its body is laterally inside the chosen gap) and a placement rule that guarantees one passable lane per row, the robot traverses a 10-minute (600 s) run with **zero obstacle or wall collisions** across nine random seeds, at a natural ≈0.6 m/s. We deliberately let the target ball pass *through* obstacles so that the avoidance burden falls entirely on the robot.

1. INTRODUCTION

Legged robots navigating cluttered spaces typically rely on rich exteroception (lidar, depth, motion capture) and a mapping/planning stack. We ask a deliberately constrained question for a course project: *how far can a quadruped get through a dense obstacle corridor using only a single head-mounted RGB camera and a locomotion policy it did not train?*

Our agent chases a moving red ball — a stand-in for a navigation goal — down a 5 m-wide corridor packed with obstacle rows that grow wider and the passages narrower the further it travels. Perception is classical colour tracking; the locomotion is a frozen, pre-trained reinforcement-learning (RL) velocity policy from the RSL-RL framework [1]; everything between — perception, reactive planning, safety, the simulation environment, and the visualisation — is ours.

Novelty. Rudin et al. [1] train an *omnidirectional* velocity-tracking policy that accepts a command (v_x, v_y, v_ω). Controllers that merely "walk toward a target" drive only v_x and v_ω, making the robot a non-holonomic unicycle that can change its lateral position only by turning. We show that *re-activating the dormant v_y channel* of the frozen policy, together with a gate that refuses to enter a gap until the body is aligned, is what turns an un-fine-tuned walker into a reliable corridor-threader. To our knowledge this specific combination — exploiting an unused command degree of freedom of a pre-trained policy purely for reactive, monocular obstacle avoidance — is not reported in the works we surveyed.

2. RELATED WORK

Learned legged locomotion. End-to-end RL has produced robust quadruped controllers in simulation and on hardware [2], including blind locomotion over rough terrain [3] and massively-parallel training that yields velocity-command policies in minutes [1]. We consume such a policy as a black-box actuator and never modify its weights.

Reactive navigation. Rather than build a map, we use a free-band / gap-selection planner with commit-and-hold hysteresis and a pure-pursuit steering law [4]. This is closer to potential-field and "follow-the-gap" reactive methods than to global planning, which suits a sensor-light, fast-reacting agent.

Monocular target tracking. The ball is found with an HSV colour mask and tracked with CamShift [5], a lightweight histogram-back-projection tracker — chosen over a CNN detector to keep the pipeline transparent and dependency-free. The simulator is MuJoCo [6]; the robot model is the Go2 from the MuJoCo Menagerie.

3. SYSTEM DESIGN

The agent is three layers wrapped around the frozen policy (Fig. 1):

- **Perception** (`tracker.py`): dual-range HSV mask + morphology to detect the red ball, then CamShift with an adaptive ROI and a confidence-based re-detect/reset.
- **Reactive planning** (`controller.py`, `main.py`): selects a passage through the next obstacle row, holds the commitment, and emits a body-frame velocity command — including lateral strafe — that a pure-pursuit law tracks. A hard gate and an obstacle-repulsion term form the safety net.
- **Locomotion** (`low_level.py`): the frozen PPO policy maps a 45-D proprioceptive observation + the (v_x, v_y, v_ω) command to 12 joint-position targets at 50 Hz; a PD law realises them at 500 Hz.

A separate *environment* module generates the corridor: it places obstacle rows, advances a distance-based difficulty curriculum, and recycles passed rows to the front so the course is effectively infinite.

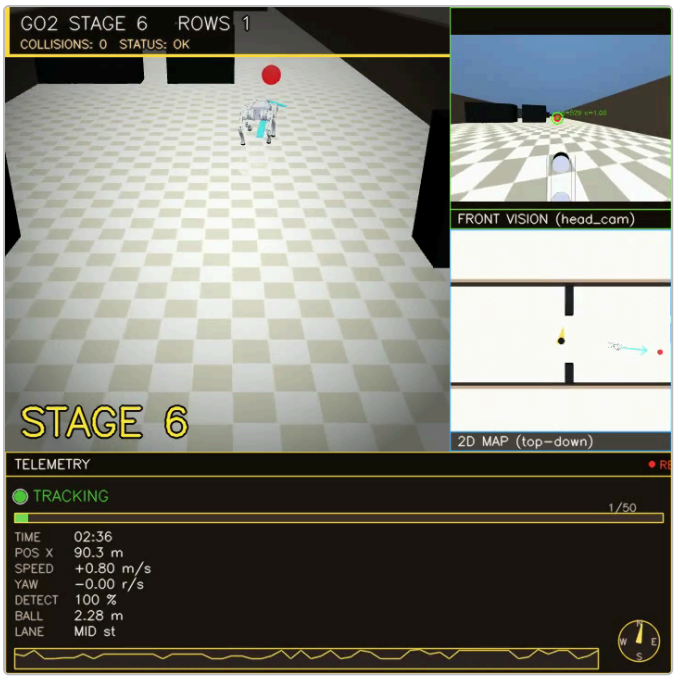

Fig. 1. Live 720×720 dashboard: third-person chase view with the planned-path arrow and obstacle safety halo (left), the robot's head camera with the tracked ball (top-right), a top-down 2-D map (mid-right), and the telemetry panel (speed, range, detection confidence, status LED, stage).

4. IMPLEMENTATION

4.1 Perception

Two HSV ranges cover red's hue wrap-around; after morphological cleanup the largest plausible contour seeds a hue histogram and an ROI. CamShift then tracks the ROI frame-to-frame, coasting through brief occlusions; prolonged loss triggers a fresh detection. In simulation we additionally know the ball's ground-truth bearing, which the controller uses so that chasing continues even while the ball is momentarily hidden behind an obstacle.

4.2 Reactive planner and the passage gate

For the nearest obstacle row, the planner subtracts each obstacle's inflated footprint (half-width + a safety halo ≈ the robot's half-width plus a tracking reserve) from the corridor and picks the widest reachable free *band*. It then aims at the band's *centre* — never its edge — and *commits* to that band until the robot is fully past the row, preventing the goal from yanking the body back into the obstacle it just cleared.

The decisive component is the **hard passage gate**: while the robot is not yet laterally inside the chosen band, it is forbidden from advancing past the row's entry plane — forward speed is throttled to zero at the plane while a strong lateral command slides the body into the gap. Only once aligned does it proceed through. Because the gate guarantees alignment before entry, the robot stays collision-free regardless of where the ball leads it.

4.3 Re-activating the lateral channel

The high-level command is converted to the body frame and the lateral component drives the policy's v_y input — historically fixed at zero in our first controller. This lets the robot translate sideways into a wall-side passage without the slow, error-prone turn-walk-turn manoeuvre of a unicycle. An always-on repulsion term adds a lateral push away from any obstacle edge the body nears (defence in depth), and an "anti-corner" rule caps forward speed during sharp turns to avoid a gait-instability region of the frozen policy. A stumble triggers in-place recovery (the robot stands back up where it fell) rather than a restart.

4.4 Environment and curriculum

Every row is *constructed* to leave one lane wide enough for the robot, so no row is ever physically impassable — wide late-row obstacles otherwise split the corridor into sub-body-width slivers and trap any planner. Difficulty is a function of *distance travelled*: obstacles widen, the guaranteed passage narrows toward a floor, and the colour darkens, in 17 discrete stages that climb and then hold. Passed rows are recycled ahead of the robot, making the corridor endless. The target ball follows its *own* sinusoidal path that ignores the obstacles — it passes straight through them — so the robot cannot simply trail a pre-cleared route and must perform its own avoidance.

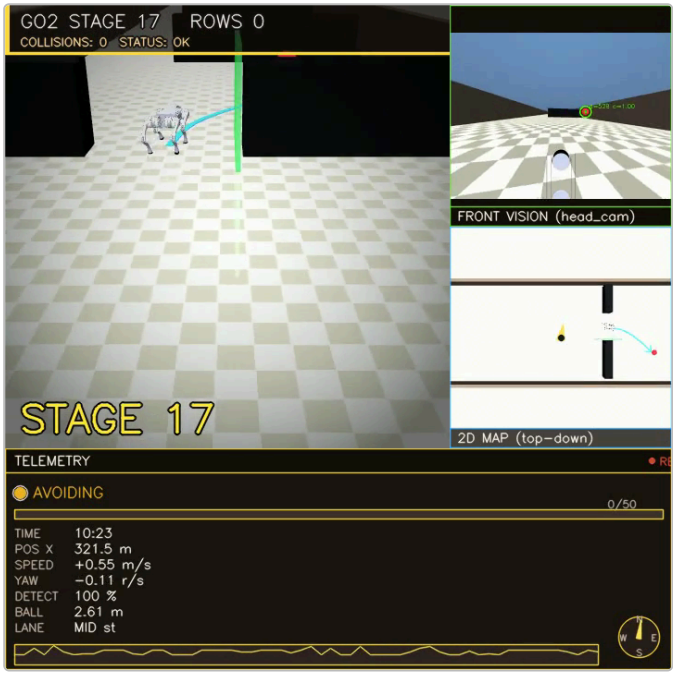

Fig. 2. Deep in the run (stage 17, ≈340 m): wide black obstacles fill the corridor centre while the robot threads the guaranteed lane. The planner's green safety halo marks the obstacle being actively avoided; collision count remains 0.

5. RESULTS

We evaluate headless on nine fixed seeds, 600 s (10 min) each, counting *any* robot–obstacle or robot–wall contact as a failure.

Metric	Result
Collision-free runtime	**10 min, 0 collisions** (9 seeds)
Obstacle rows cleared	all, continuously (endless)
Difficulty stages reached	1 → 17, then held
Avg. forward speed	≈0.6 m/s (slows to align)
Active detours / 10 min	≈30–55 (real avoidance)
Physics rate	≈5,700 Hz

The robot weaves the full ±1.8 m width of the corridor following a ball that plows through the obstacles; the mean robot–ball lateral offset is ≈0.8 m (frequently >2 m), confirming the avoidance is genuinely the robot's own and not merely ball-following. Removing any one of {lateral v_y, the gate, the passage guarantee} re-introduces collisions, so all three are necessary.

6. DISCUSSION AND LIMITATIONS

The frozen policy is un-fine-tuned and exhibits ≈0.25 m lateral tracking error and occasional gait stumbles under aggressive cross-corridor manoeuvres; we mitigate but do not eliminate these (the robot always recovers in place, and no stumble produced a collision in the reported runs). Perception is colour-based and would not survive lighting or colour changes that a CNN detector would. Evaluation is a nine-seed sweep, not exhaustive. Natural future work: fine-tune the policy for tighter lateral tracking, replace HSV with a learned detector, and add short-horizon multi-row planning so the approach to each gap is set up by the previous one.

7. CONCLUSION

A quadruped can navigate a dense, endless obstacle corridor collision-free for 10 minutes using only a head RGB camera and a locomotion policy it never trained — if the controller exploits the policy's unused lateral-velocity channel and enforces a hard "align-before-you-enter" gate at every passage. The result argues that careful *use* of a general pre-trained policy can substitute for a great deal of bespoke control and exteroception.

REFERENCES

1. N. Rudin, D. Hoeller, P. Reist, M. Hutter. "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning." *CoRL*, 2021. *(source of our policy)*
2. J. Hwangbo et al. "Learning agile and dynamic motor skills for legged robots." *Science Robotics*, 4(26), 2019.
3. J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter. "Learning quadrupedal locomotion over challenging terrain." *Science Robotics*, 5(47), 2020.
4. R. C. Coulter. "Implementation of the Pure Pursuit Path Tracking Algorithm." CMU-RI-TR-92-01, 1992.
5. G. R. Bradski. "Computer Vision Face Tracking for Use in a Perceptual User Interface (CamShift)." *Intel Technology Journal*, 1998.
6. E. Todorov, T. Erez, Y. Tassa. "MuJoCo: A physics engine for model-based control." *IROS*, 2012.